

RESUMECRAFT

Java Resume Builder

PROJECT REPORT

A Java console application for creating, managing, formatting and exporting professional resumes.

Project	ResumeCraft
Technology	Java • Maven
Interface	Console / menu driven
Output	TXT and HTML
Report	Requirements, architecture, design, implementation, testing and challenges

Prepared from the current ResumeCraft Java project implementation.

1. Introduction

ResumeCraft is a Java-based resume builder designed to guide a user through the creation of a structured professional resume. The current implementation collects personal information, education, projects and skills, stores them in a central Resume object, provides menu-driven management, and exports the formatted result to TXT and HTML files.

The project separates resume data from service operations, formatting and file writing. This organization demonstrates object-oriented programming and makes the application easier to maintain.

Objectives

- Provide a guided process for entering resume information.
- Allow users to view, edit, delete and add resume entries.
- Validate important user input.
- Format the stored resume into a readable professional representation.
- Generate usable TXT and HTML output files.

2. Problem Statement

Creating a professional resume manually can involve repeated editing and formatting. ResumeCraft treats the problem as a structured Java application task: collect information, validate it, store it in memory, manipulate it through defined operations, format it, and generate output files.

Problem

Design and implement a Java application that guides a user through resume creation and provides controlled operations for viewing, editing, deleting and adding entries, followed by export to TXT and HTML.

Scope

- Personal information: name, email, phone, location, LinkedIn, GitHub and professional summary.
- Education: multiple entries containing degree, institution, location, start/end years and grade.
- Projects: multiple entries containing project name, technologies, description and GitHub link.
- Skills: multiple entries containing name, category and proficiency.
- Menu-based management after initial creation.
- File-based output; no relational database is used.

3. Functional Requirements

ID	Requirement	Implementation
FR-01	Create personal information	PersonalInfo is created and stored through ResumeService.
FR-02	Add education	User chooses a number of education entries and adds them to Resume.
FR-03	Add projects	Project entries are collected and added through the service.
FR-04	Add skills	Skill entries are collected and added through the service.
FR-05	View resume	The current Resume is formatted and displayed.
FR-06	Edit resume	Personal information, education, projects and skills can be edited.
FR-07	Delete entries	Education, projects and skills can be deleted by index.
FR-08	Add more entries	The main menu provides an Add More Entries operation.
FR-09	Save resume	ResumeFileWriter writes files inside the output directory.
FR-10	Export	The application produces TXT and HTML resume output.

4. Non-functional Requirements

- **Usability:** Clear headings and numbered menus guide the user through the console workflow.
- **Input reliability:** Required fields reject empty values; email, phone, year and numeric values are validated.
- **Maintainability:** Model, service, formatter and file-writing responsibilities are separated.
- **Consistency:** The same central Resume object is used for viewing, editing, deleting, formatting and saving.
- **Portability:** The project uses a Maven-style Java structure and project-relative output files.
- **Extensibility:** The template-enabled writer supports MODERN, CLASSIC and MINIMAL HTML layouts.

Validation rules

- Required text fields are repeatedly requested until non-empty.
- Email is checked using a regular expression.
- Phone is checked using the supported Indian-number pattern.
- Years must be four digits and an end year cannot be earlier than the start year.
- Numeric menu choices and entry counts are restricted to valid ranges.

5. System Architecture

The project follows a simple layered organization. Main.java coordinates the console workflow. ResumeService manages the Resume model. ResumeFormatter converts resume data into formatted text. ResumeFileWriter creates the output files. The model package contains the resume data classes.

Component	Responsibility
User / Console	Provides resume information and chooses menu operations.
Main.java	Controls input, validation, menus, editing, deletion and saving.
ResumeService	Provides get/set/add/remove operations around the Resume object.
Model	Stores PersonalInfo, Education, Experience, Project and Skill data.
ResumeFormatter	Builds the structured text representation of the resume.
ResumeFileWriter	Writes TXT output and generates HTML output.
ResumeTemplate	Represents MODERN, CLASSIC and MINIMAL template choices in the template-enabled writer.
Output	Generated resume.txt and HTML resume files under output/.

Architecture flow: User → Main.java → ResumeService → Resume Model → ResumeFormatter → ResumeFileWriter → TXT / HTML Output.

Figure 1. High-level system architecture.

6. Design Decisions & Rationale

Decision	Rationale
Separate model classes	Keeps different resume concepts organized and easier to manipulate.
ResumeService	Centralizes resume operations instead of putting all data manipulation directly in Main.
ArrayList collections	Supports a variable number of education, experience, project and skill entries.
Console menus	Provides a straightforward interface for demonstrating Java control flow, methods and validation.
File output	Produces usable resume artifacts without requiring a database.
Template support	Allows HTML presentation to change while keeping the same resume data.

Central design principle: Resume remains the source of current data, while services and formatters operate on that data.

7. Design Diagrams

7.1 Use Case Diagram

Actor	Use Cases
User	Create Resume • View Resume • Edit Resume • Delete Entries • Add More Entries • Save Resume

The user is the primary actor and interacts with the system through the console to create and manage the resume.

Figure 2. Use-case specification for the current application.

7.2 Workflow Diagram

Start → **Personal Information** → **Education** → **Projects** → **Skills** → **Resume Created** → **Save** → **Main Menu** → **View / Edit / Delete / Add / Save** → **Exit**

Figure 3. Main workflow from initial data collection to output.

7.3 Sequence Diagram

Typical save sequence: **User** selects **Save** → **Main** accesses the current Resume → **ResumeFormatter** creates formatted text → **ResumeFileWriter** writes TXT and HTML → control returns to the menu or exits.

Figure 4. Save-operation sequence.

7.4 Class / Component Diagram

Class	Relationship / Responsibility
Resume	Contains PersonalInfo and lists of Education, Experience, Project and Skill; provides add/remove methods.
ResumeService	Owns a Resume instance and exposes get/set/add/remove operations.
ResumeFormatter	Formats Resume data into a structured text representation.
ResumeFileWriter	Writes text and HTML files and handles template-based HTML generation.
ResumeTemplate	Defines MODERN, CLASSIC and MINIMAL template choices.
Main	Coordinates user interaction and calls service/formatter/writer operations.

Component flow: Main → ResumeService → Resume; Resume → Formatter; Formatter → FileWriter; FileWriter → selected HTML template.

7.5 ER Diagram (if storage used)

Not applicable to the current project. No relational database is used; resume information is represented by Java objects and collections during execution and exported to files.

8. Implementation Details

Project structure: The project uses a Maven-style Java layout with source code under `src/main/java/com/resumecraft`, generated output under `output/`, and a `pom.xml` build descriptor.

Resume model: Resume contains `PersonalInfo` plus lists for Education, Experience, Project and Skill. The constructor initializes these lists with `ArrayList`.

Service layer: `ResumeService` provides `getResume()`, `setPersonalInfo()`, `addEducation()`, `addExperience()`, `addProject()`, `addSkill()` and corresponding removal methods.

User interaction: `Main.java` collects personal information, multiple education entries, projects and skills, then opens a menu with View, Edit, Delete, Add More, Save and Save & Exit options.

Validation: Helper methods validate non-empty input, email, phone, years and numeric ranges. Editing uses optional-input methods so pressing Enter can preserve an existing value.

Formatting and HTML: `ResumeFormatter` creates formatted text. The HTML writer recognizes sections such as Summary, Education, Projects and Skills and produces styled HTML.

File output: `ResumeFileWriter` creates the output directory when necessary, writes `resume.txt` and generates HTML. The template-enabled version selects MODERN, CLASSIC or MINIMAL through `ResumeTemplate`.

9. Screenshots / Results

The current implementation produces the following observable results during execution:

- A welcome heading identifies ResumeCraft as a Java Resume Builder.
- Personal information, education, projects and skills are collected through prompts.
- Successful operations display confirmation messages.
- The main menu provides View Resume, Edit Resume, Delete Entries, Add More Entries, Save Resume and Save Resume & Exit.
- Saving produces a text resume and HTML resume under the output directory.
- The template-enabled writer can generate modern, classic or minimal HTML layouts.

Expected output artifacts

Artifact	Purpose
output/resume.txt	Plain-text formatted resume.
output/resume.html	Modern HTML resume in the basic writer version.
output/resume-modern.html	Modern template output in the template-enabled writer.
output/resume-classic.html	Classic template output when CLASSIC is selected.
output/resume-minimal.html	Minimal template output when MINIMAL is selected.

Exact template filenames depend on which ResumeFileWriter version is active in the project.

10. Testing Approach

Test	Action	Expected Result
T01	Valid personal information	Accepted and stored.
T02	Empty required field	Rejected; user is asked again.
T03	Invalid email	Rejected by email validation.
T04	Invalid phone	Rejected by phone validation.
T05	End year before start year	Rejected.
T06	View Resume	Current formatted resume is displayed.
T07	Edit project or skill	Selected object is updated and saved.
T08	Delete and confirm yes	Entry is removed and saved.
T09	Delete and answer no	Deletion is cancelled.
T10	Save Resume	TXT and HTML files are written.
T11	Invalid menu number	Out-of-range choice is rejected.

11. Challenges Faced

- **Input validation:** Incorrect values must be handled without accepting unusable data or terminating the workflow.
- **Variable-length data:** Education, projects and skills can occur multiple times, requiring list-based management.
- **Editing safely:** Optional input allows existing values to remain unchanged when the user presses Enter.
- **Separation of concerns:** Resume data is kept independent from formatting and file generation.
- **HTML generation:** Structured text has to be interpreted into sections and safely inserted into HTML.
- **File handling:** The output directory must be created when needed and write failures must be handled.

Lessons learned

- Use small classes with clear responsibilities.
- Validate input at the point where it enters the application.
- Keep Resume as the central source of current data.
- Separate formatting from data management.
- Use Git to track source code and documentation while excluding generated build artifacts.

12. Conclusion

ResumeCraft demonstrates how core Java programming concepts can be combined into a complete application. Classes and objects model resume information, collections support multiple entries, methods organize behavior, validation improves reliability, menu-driven control flow manages interaction, and file I/O produces usable output artifacts.

The project also demonstrates separation between model, service, formatting and output responsibilities. The current implementation is suitable for a Java programming project and provides a foundation for future extensions such as persistent storage, additional resume sections, richer templates, PDF generation or a graphical interface.

Final project summary

Area	Current Implementation
Language	Java
Build	Maven-style project structure
Interface	Console / menu driven
Data	In-memory Java objects and lists
Management	View, edit, delete and add operations
Validation	Required fields, email, phone, years and numeric ranges
Export	TXT and HTML
Templates	Modern, Classic and Minimal in template-enabled writer
Database	Not used

End of Project Report